Advanced information, computation, communication I
EPFL - Fall semester 2026-2027

Week 8

November 6, 2026

For the sake of precision and rigor, we mean a Big-Theta function of n when talking complexity in all exercices of this homework. In real world cases, Big-O is often sufficient when describing algorithmic complexity.

Exercise 1.

1. Show that $5x$ is $o(x^2)$.
2. Show that $2x^2$ is not $o(x^2)$.
3. Show that $1/x$ is $o(x)$.

Exercise 2. Let f be an arbitrary function from $\mathbf{N}$ to $\mathbf{R}_{>0}$.

Let g_1, g_2 be two functions from $\mathbf{N}$ to $\mathbf{R}_{>0}$ such that g_1 and g_2 are both $\Theta(f)$.

1. Show that the function $g_1 + g_2$ is $\Theta(f)$ or provide a counterexample.
2. Show that the function $g_1 g_2$ is $\Theta(f^2)$ or provide a counterexample.

Let g_3, g_4 be two functions from $\mathbf{N}$ to $\mathbf{R}$ such that g_3 and g_4 are both $\Theta(f)$.

3. Show that the function $g_3 + g_4$ is $\Theta(f)$ or provide a counterexample.
4. Show that the function $g_3 g_4$ is $\Theta(f^2)$ or provide a counterexample.

Let g be a function from $\mathbf{N}$ to $\mathbf{R}_{>0}$ such that g is $O(f)$.

5. Show that 2^g is $O(2^f)$, or provide a counterexample.

Exercise 3. Consider the two statements below, where k and ℓ are constants with $k > \ell \geq 2$ and $m \to \infty$:

$$\log_m(k) \text{ is } \Theta(\log_m(\ell)) \qquad k^{\log_\ell(m)} \text{ is } O(\ell^{\log_k(m)}).$$

- ◯ They are both false.
- ◯ Only the first is true.
- ◯ Only the second is true.
- ◯ They are both true.

Exercise 4. *Consider the following two statements:*

$$\Big(f \text{ is } o(f)\Big) \quad \text{and} \quad \Big(f \text{ is } o(g) \text{ implies } f \text{ is } O(g)\Big).$$

- ◯ *Only the second is true.*
- ◯ *They are both false.*
- ◯ *Only the first is true.*

◯ *They are both true.*

Exercise 5. *Given the two statements below, where $d > 0$ is an integer constant and a_i for all $i \in \mathbf{Z}$ are positive integers with $\max_{i \in \mathbf{Z}}(a_i) = D$ for a constant $D > 0$,*

$$\sum_{i=0}^{n} a_i i^d \text{ is } \Theta(n^{d+1}) \qquad \sum_{i=0}^{d} a_i n^i \text{ is } \Theta(n^d)$$

◯ *They are both true.*

◯ *Only the first is true.*

◯ *Only the second is true.*

◯ *They are both false.*

Exercise 6. *Counting the number of swap and the number of comparisons, what is the time complexity of the following algorithm (as a Big-Theta function of n):*

Algorithm 1 Selection Sort

```
1:  for i = 1 to n − 1 do
2:      min ← i + 1
3:      for j = i + 1 to n do
4:          if a_min > a_j then
5:              min ← j
6:          end if
7:      end for
8:      if a_i > a_min then
9:          swap a_i and a_min
10:     end if
11: end for
```

Exercise 7. *Determine the time complexity of algorithm 2.*

Algorithm 2 Matrix Multiplication

Require: Matrices $A(n \times n)$ and $B(n \times n)$
Ensure: Result matrix $C(n \times n)$ product of the two input matrices

```
1:  for i ← 1 to n do
2:      for j ← 1 to n do
3:          sum ← 0
4:          for k ← 1 to n do
5:              sum ← sum + A[i, k] × B[k, j]
6:          end for
7:          C[i, j] ← sum
8:      end for
9:  end for
10: return C
```

1. *Counting both additions and multiplications, what is the time complexity of algorithm 2 (as a Big-Theta function of n)?*
2. *Counting only multiplications, what is the time complexity of algorithm 2 (as a Big-Theta function of n)?*

Exercise 8. *Counting the number of comparisons, what is the time complexity of the algorithm 3?*

Algorithm 3 Determines where there are two elements in an array that have a target sum

Require: arr is an array (indexed starting at 0) assumed to be sorted and $target$ is a number
Ensure: Returns if there exists two elements in the array arr such that their sum is the $target$.

```
1: n ← length(arr)
2: for i ← 0 to n − 1 do
3:     for j ← 0 to n − 1 do
4:         if arr[i] + arr[j] == target then
5:             return TRUE
6:         end if
7:     end for
8: end for
9: return FALSE
```

Exercise 9. *Counting the number of comparisons, what is the time complexity of the algorithm 4? Note that the function binarySearch has $log_2(n)$ time complexity (counting the number of comparisons).*

Algorithm 4 Determines where there are two elements in an array that have a target sum, more sophisticated

Require: arr is an array (indexed starting at 0) assumed to be sorted and $target$ is a number
Ensure: Returns if there exists two elements in the array arr such that their sum is the target.

```
1: n ← length(arr)
2: for i ← 0 to n − 1 do
3:     if binarySearch(target − arr[i], arr) ! = −1 then
4:         return TRUE
5:     end if
6: end for
7: return FALSE
```